American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Introduction to Data Science

Mid-Term Project Report

Spring 2025-2026

Section: F

Group: 05

Project Title: Student Performance Analysis

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: TAMZID IQBAL ID: 22-46255-1	Examination and Comparison of Variability Across Categories, Normalization of Continuous Attribute, Splitting the Dataset for Training and Testing, Feature Selection.
2.	Name: ANIKA TABASSUM RIA ID: 22-49586-3	Loading and Understanding the Dataset, Checking Missing Data (NA values), Filling Missing Values, Conversion of Imbalanced Dataset into Balanced Dataset using Oversampling.
3.	Name: FAHIM FAISAL SHIFAT ID: 22-49603-3	Data Exploration, Checking and Fixing Invalid or Noisy Values, Checking and Removing Duplicate Data, Descriptive Statistics According to Target Classes.
4.	Name: MALIHA TUSNIM PRAPTI ID: 22-49917-3	Checking Outliers and Fixing It, Conversion of Attributes from Numeric to Categorical, Comparison of Average Values Across Two Categories, Filtering the Data.

Dataset Description

The selected dataset is the **Student Performance Dataset** collected from Kaggle. It contains various features related to students' demographic information, study habits, parental involvement, and extracurricular activities. The dataset includes both numerical features such as Age, StudyTimeWeekly, Absences, and GPA, and categorical features such as Gender, Ethnicity, ParentalEducation, Tutoring, and ParentalSupport.

The target variable of this dataset is **GradeClass**, which represents students' academic performance categories based on their GPA. It is a categorical variable with multiple classes such as A, B, C, D, and F. This dataset is suitable for analyzing the relationship between different features and students' academic performance.

The detailed description of the attributes used in this dataset is provided below:

1. Attribute Details:

Student ID

- **StudentID**: A unique identifier assigned to each student (1001 to 3392).

Demographic Details

- **Age**: The age of the students ranges from 15 to 18 years.
- **Gender**: Gender of the students, where 0 represents Male and 1 represents Female.
- **Ethnicity**: The ethnicity of the students, coded as follows: o 0: Caucasian o 1: African American o 2: Asian o 3: Other
- **Parental Education**: The education level of the parents, coded as follows:
  - 0: None
  - 1: High School
  - 2: Some College
  - 3: Bachelor's
  - 4: Higher

Study Habits:

- **StudyTimeWeekly**: Weekly study time in hours, ranging from 0 to 20.
- **Absences**: Number of absences during the school year, ranging from 0 to 30.
- **Tutoring**: Tutoring status, where 0 indicates No and 1 indicates Yes.

Parental Involvement

- **Parental Support**: The level of parental support, coded as follows:
  - 0: None

- 1: Low
- 2: Moderate
- 3: High
- 4: Very High

Extracurricular Activities

- **Extracurricular**: Participation in extracurricular activities, where 0 indicates No and 1 indicates Yes.
- **Sports**: Participation in sports, where 0 indicates No and 1 indicates Yes.
- **Music**: Participation in music activities, where 0 indicates No and 1 indicates Yes.
- **Volunteering**: Participation in volunteering, where 0 indicates No and 1 indicates Yes.

Academic Performance

- **GPA**: Grade Point range is from 0 to 4.

Target Variable: Grade Class

- **Grade Class**: Classification of students' grades based on GPA:
  - 0: 'A' (GPA >= 3.5)
  - 1: 'B' (3.0 <= GPA < 3.5)
  - 2: 'C' (2.5 <= GPA < 3.0)
  - 3: 'D' (2.0 <= GPA < 2.5)
  - 4: 'F' (GPA < 2.0)

Dataset Modification Details:

Missing Values	
Row Number	**Attributes**
22	Absences
26	Age, Extracurricular
42	Absences
73	Gender
80	GPA

114	Sports
116	ParentalSupport
180	Tutoring
286	ParentalEducation
400	ParentalEducation
618	Age
Outlier	
137	Age(150)
635	Absences(200)
317	Age(30)
Invalid Data	
556	Ethnicity(9)
71	Ethnicity(other)
483	Ethnicity(Asian)
37	Tutoring(11)

A Column called “Height” is added in the dataset which has no relation with student performance. 16 and 17 number rows are duplicate.

Project Implementation Detail

Title of the task1: Loading and Understanding the Dataset

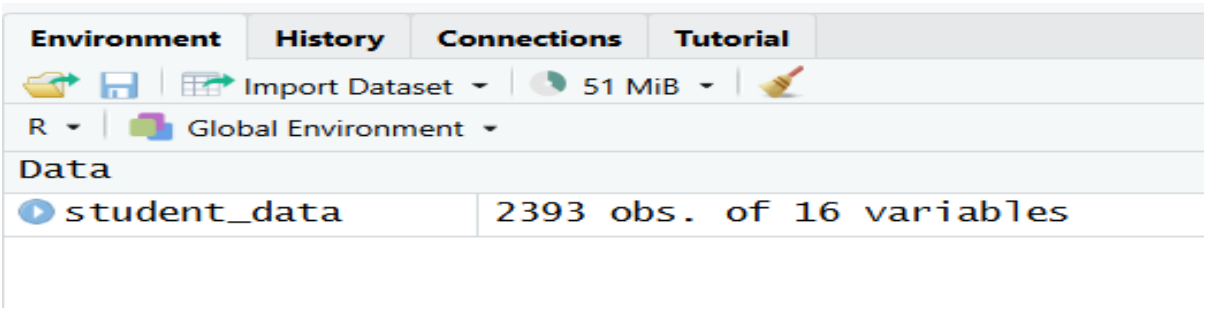
Description of the task1: In this task, the dataset is loaded into R using the read.csv() function. After loading the dataset, basic functions such as head() and str() are used to understand the dataset by viewing sample data and examining the structure and data types of the attributes.

Code for task1: student_data <- read.csv("ids_mid_project_group_05.csv")

head(student_data)

str(student_data)

Sample output of code for task1 :




```
> head(student_data)
  StudentID Age Gender Ethnicity Height ParentalEducation StudyTimeWeekly Absences Tutoring ParentalSupport
1      1001  17      1         0    152                 2       19.833723        7        1               2
2      1002  18      0         0    158                 1       15.408756        0        0               1
3      1003  15      0         2    165                 3        4.210570       26        0               2
4      1004  17      1         0    172                 3       10.028829       14        0               3
5      1005  17      1         0    160                 2        4.672495       17        1               3
6      1006  18      0         0    149                 1        8.191219        0        0               1
  Extracurricular Sports Music Volunteering       GPA GradeClass
1               0      0     1            0 2.9291956          2
2               0      0     0            0 3.0429148          1
3               0      0     0            0 0.1126023          4
4               1      0     0            0 2.0542181          3
5               0      0     0            0 1.2880612          4
6               1      0     0            0 3.0841836          1
>
```

```
> str(student_data)
'data.frame':   2393 obs. of  16 variables:
 $ StudentID         : int  1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 ...
 $ Age               : int  17 18 15 17 17 18 15 15 17 16 ...
 $ Gender            : int  1 0 0 1 1 0 0 1 0 1 ...
 $ Ethnicity         : chr  "0" "0" "2" "0" ...
 $ Height            : int  152 158 165 172 160 149 168 174 155 161 ...
 $ ParentalEducation: int  2 1 3 3 2 1 1 4 0 1 ...
 $ StudyTimeWeekly   : num  19.83 15.41 4.21 10.03 4.67 ...
 $ Absences          : int  7 0 26 14 17 0 10 22 1 0 ...
 $ Tutoring          : int  1 0 0 0 1 0 0 1 0 0 ...
 $ ParentalSupport   : int  2 1 2 3 3 1 3 1 2 3 ...
 $ Extracurricular   : int  0 0 0 1 0 1 0 1 0 1 ...
 $ Sports            : int  0 0 0 0 0 0 1 0 1 0 ...
 $ Music             : int  1 0 0 0 0 0 0 0 0 0 ...
 $ Volunteering      : int  0 0 0 0 0 0 0 0 1 0 ...
 $ GPA               : num  2.929 3.043 0.113 2.054 1.288 ...
 $ GradeClass        : int  2 1 4 3 4 1 2 4 2 0 ...
>
```

Code description of task1: The dataset is loaded into R using the read.csv() function by providing the file path. The head() function is used to display the first few rows of the dataset, which helps to get an initial idea about the data. The str() function is used to examine the

structure of the dataset and identify the data types of each attribute, such as numeric and categorical variables.

Title of the task2: Data Exploration

Description of the task2: In this task, the dataset is explored to understand its overall characteristics. Basic functions are used to examine the dataset, including summary statistics, attribute names, and dataset dimensions.

Code for task2: summary(student_data)

dim(student_data)

names(student_data)

Sample output of code for task2:

```
> summary(student_data)
   StudentID          Age              Gender         Ethnicity             Height      ParentalEducation
 Min.   :1001   Min.   : 15.00   Min.   :0.0000   Length:2393        Min.   :140.0   Min.   :0.000
 1st Qu.:1598   1st Qu.: 15.00   1st Qu.:0.0000   Class :character   1st Qu.:156.0   1st Qu.:1.000
 Median :2196   Median : 16.00   Median :1.0000   Mode  :character   Median :169.0   Median :2.000
 Mean   :2196   Mean   : 16.53   Mean   :0.5113                      Mean   :169.8   Mean   :1.746
 3rd Qu.:2794   3rd Qu.: 17.00   3rd Qu.:1.0000                      3rd Qu.:184.0   3rd Qu.:2.000
 Max.   :3392   Max.   :150.00   Max.   :1.0000                      Max.   :200.0   Max.   :4.000
                NA's   :2        NA's   :1                                           NA's   :2
 StudyTimeWeekly       Absences         Tutoring       ParentalSupport Extracurricular      Sports
 Min.   : 0.001056   Min.   :  0.00   Min.   : 0.0000   Min.   :0.000   Min.   :0.0000   Min.   :0.0000
 1st Qu.: 5.043640   1st Qu.:  7.00   1st Qu.: 0.0000   1st Qu.:1.000   1st Qu.:0.0000   1st Qu.:0.0000
 Median : 9.705614   Median : 15.00   Median : 0.0000   Median :2.000   Median :0.0000   Median :0.0000
 Mean   : 9.772588   Mean   : 14.62   Mean   : 0.3056   Mean   :2.122   Mean   :0.3829   Mean   :0.3035
 3rd Qu.:14.408321   3rd Qu.: 22.00   3rd Qu.: 1.0000   3rd Qu.:3.000   3rd Qu.:1.0000   3rd Qu.:1.0000
 Max.   :19.978094   Max.   :200.00   Max.   :11.0000   Max.   :4.000   Max.   :1.0000   Max.   :1.0000
                     NA's   :2        NA's   :1         NA's   :1       NA's   :1        NA's   :1
     Music          Volunteering         GPA          GradeClass
 Min.   :0.0000   Min.   :0.0000   Min.   :0.000   Min.   :0.000
 1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:1.175   1st Qu.:2.000
 Median :0.0000   Median :0.0000   Median :1.893   Median :4.000
 Mean   :0.1968   Mean   :0.1571   Mean   :1.906   Mean   :2.984
 3rd Qu.:0.0000   3rd Qu.:0.0000   3rd Qu.:2.622   3rd Qu.:4.000
 Max.   :1.0000   Max.   :1.0000   Max.   :4.000   Max.   :4.000
                                   NA's   :1
```

```
> dim(student_data)
[1] 2393   16
>
```

```
> names(student_data)
 [1] "StudentID"        "Age"              "Gender"           "Ethnicity"        "Height"
 [6] "ParentalEducation" "StudyTimeWeekly"  "Absences"         "Tutoring"         "ParentalSupport"
[11] "Extracurricular"  "Sports"           "Music"            "Volunteering"     "GPA"
[16] "GradeClass"
>
```

Code description of task2: The dataset is explored using basic functions. The summary() function provides descriptive statistics for numerical attributes. The dim() function is used to determine the number of rows and columns in the dataset. The names() function displays the attribute names. This step helps to understand the dataset before applying further data preparation and cleaning techniques.

Title of the task3: Checking Missing Data (NA values)

Description of the task3: In this task, missing values in the dataset are identified. The is.na() function is used along with colSums() to count the number of missing values in each attribute. This helps to determine which attributes contain missing data and require further processing.

Code for task3: colSums(is.na(student_data))

Sample output of code for task3:

```
> colSums(is.na(student_data))
       StudentID              Age           Gender        Ethnicity          Height ParentalEducation
               0                2                1                0                0                2
 StudyTimeWeekly         Absences         Tutoring  ParentalSupport   Extracurricular           Sports
               0                2                1                1                1                1
           Music      Volunteering              GPA       GradeClass
               0                0                1                0
> |
```

Code description of task3: The is.na() function is used to identify missing values in the dataset, and the colSums() function counts the number of missing values in each attribute. The output shows that some attributes such as StudentID, Ethnicity, StudyTimeWeekly, Music, Volunteering, and GradeClass do not contain any missing values. However, several attributes including Age, Gender, ParentalEducation, Absences, Tutoring, ParentalSupport, Extracurricular, Sports, and GPA contain missing values. This information helps in determining which attributes require handling of missing data in the next step.

Title of the task4: Filling Missing Values

Description of the task4: Missing values are replaced using mean for numerical attributes and mode for categorical attributes.

Summary of missing values in Dataset and solution method:

Missing value attributes	Type	Method
Age	Numerical	Mean
Gender	Categorical	Mode
ParentalEducation	Categorical	Mode
Absences	Numerical	Median(Many students have few absences, few have very high)
Tutoring	Categorical	Mode
ParentalSupport	Categorical	Mode
Extracurricular	Categorical	Mode
Sports	Categorical	Mode
GPA	Numerical	Mean

Code for task4: **For Age:** student_data$Age[is.na(student_data$Age)] <-
mean(student_data$Age, na.rm = TRUE)
student_data$Age <- as.integer(round(student_data$Age))
sum(is.na(student_data$Age))

For Gender: mode_gender <- names(which.max(table(student_data$Gender)))

mode_gender

student_data$Gender[is.na(student_data$Gender)] <- mode_gender

sum(is.na(student_data$Gender))

For ParentalEducation:

mode_parent_edu <-names(which.max(table(student_data$ParentalEducation)))

mode_parent_edu

student_data$ParentalEducation[is.na(student_data$ParentalEducation)] <- mode_parent_edu

sum(is.na(student_data$ParentalEducation))

For Absence: med_absence <- median(student_data$Absences, na.rm = TRUE)

med_absence

student_data$Absences[is.na(student_data$Absences)] <- med_absence

sum(is.na(student_data$Absences))

For Tutoring: mode_tutoring <- names(which.max(table(student_data$Tutoring)))

mode_tutoring

student_data$Tutoring[is.na(student_data$Tutoring)] <- mode_tutoring

sum(is.na(student_data$Tutoring))

For ParentalSupport:

mode_parent_sup <- names(which.max(table(student_data$ParentalSupport)))

mode_parent_sup

student_data$ParentalSupport[is.na(student_data$ParentalSupport)] <- mode_parent_sup

sum(is.na(student_data$ParentalSupport))

For Extracurricular: mode_eca <- names(which.max(table(student_data$Extracurricular)))

mode_eca

student_data$Extracurricular[is.na(student_data$Extracurricular)] <- mode_eca

sum(is.na(student_data$Extracurricular))

For Sports: mode_sports <- names(which.max(table(student_data$Sports)))

mode_sports

student_data$Sports[is.na(student_data$Sports)] <- mode_sports

sum(is.na(student_data$Sports))

For GPA: mean_gpa <- mean(student_data$GPA, na.rm = TRUE)

mean_gpa

student_data$GPA[is.na(student_data$GPA)] <- mean_gpa

sum(is.na(student_data$GPA))

Sample output of code for task4:

For Age:

```
> student_data$Age[is.na(student_data$Age)] <- mean(student_data$Age, na.rm = TRUE)
> student_data$Age <- as.integer(round(student_data$Age))
> sum(is.na(student_data$Age))
[1] 0
```

For Gender:

```
> mode_gender <- names(which.max(table(student_data$Gender)))
> mode_gender
[1] "1"
> student_data$Gender[is.na(student_data$Gender)] <- mode_gender
> sum(is.na(student_data$Gender))
[1] 0
>
```

For Parental Education:

```
> mode_parent_edu <- names(which.max(table(student_data$ParentalEducation)))
> mode_parent_edu
[1] "2"
> student_data$ParentalEducation[is.na(student_data$ParentalEducation)] <- mode_parent_edu
> sum(is.na(student_data$ParentalEducation))
[1] 0
>
```

For Absence:

```
> med_absence <- median(student_data$Absences, na.rm = TRUE)
> med_absence
[1] 15
> student_data$Absences[is.na(student_data$Absences)] <- med_absence
> sum(is.na(student_data$Absences))
[1] 0
>
```

For Tutoring:

```
> mode_tutoring <- names(which.max(table(student_data$Tutoring)))
> mode_tutoring
[1] "0"
> student_data$Tutoring[is.na(student_data$Tutoring)] <- mode_tutoring
> sum(is.na(student_data$Tutoring))
[1] 0
>
```

For Parental Support:

```
> mode_parent_sup <- names(which.max(table(student_data$ParentalSupport)))
> mode_parent_sup
[1] "2"
> student_data$ParentalSupport[is.na(student_data$ParentalSupport)] <- mode_parent_sup
> sum(is.na(student_data$ParentalSupport))
[1] 0
>
```

For Extracurricular:

```
> mode_eca <- names(which.max(table(student_data$Extracurricular)))
> mode_eca
[1] "0"
> student_data$Extracurricular[is.na(student_data$Extracurricular)] <- mode_eca
> sum(is.na(student_data$Extracurricular))
[1] 0
>
```

For Sports:

```
> mode_sports <- names(which.max(table(student_data$Sports)))
> mode_sports
[1] "0"
> student_data$Sports[is.na(student_data$Sports)] <- mode_sports
> sum(is.na(student_data$Sports))
[1] 0
>
```

For GPA:

```
> mean_gpa <- mean(student_data$GPA, na.rm = TRUE)
> mean_gpa
[1] 1.906297
> student_data$GPA[is.na(student_data$GPA)] <- mean_gpa
> sum(is.na(student_data$GPA))
[1] 0
>
```

Code description of task4: The code handles missing values in the dataset by applying different methods based on the type of each attribute. For the Age attribute, missing values are replaced using the mean value, and then the values are rounded and converted into integers to keep the attribute in a proper numeric form. For the categorical attributes Gender, ParentalEducation, Tutoring, ParentalSupport, Extracurricular, and Sports, the mode (most frequent category) is calculated and used to replace missing values. For Absences, the median is used instead of the mean because this attribute may contain comparatively high values, and the median is more appropriate in such cases. For GPA, missing values are replaced using the mean value. After handling each attribute, the sum(is.na()) function is used to check whether any missing values remain. The outputs show 0 for all these attributes, which confirms that all missing values have been successfully handled.

Title of the task5: Checking and Removing Duplicate Data

Description of the task5: In this task, duplicate rows in the dataset are identified and removed. Duplicate data can negatively affect analysis results, so it is important to detect and eliminate such records to maintain data quality.

Code for task5: Checking Duplicate: anyDuplicated(student_data)

Delete Duplicate: student_data <- student_data[!duplicated(student_data),]

Sample output of code for task5:

```
> anyDuplicated(student_data)
[1] 16
>
```

13	1013	17	0	1	1	10.038711620	21	0	3	1	0
14	1014	17	0	1	2	12.101425070	21	0	4	0	1
15	1015	18	1	0	1	11.197810640	9	1	2	0	0
17	1016	15	0	0	2	9.728100711	17	1	0	0	1
18	1017	18	0	3	1	10.098656080	14	0	2	1	1

Code description of task5: The anyDuplicated() function is used to identify the position of duplicate rows in the dataset. The output shows that row number 16 is a duplicate. To remove duplicate rows, the duplicated() function is used, and only unique rows are retained in the dataset. After applying this method, the duplicate row is successfully removed, improving the overall quality of the dataset.

Title of the task6: Checking Outliers and Fixing It

Description of the task6: In this task, outliers are identified and handled in the numerical attributes of the dataset using the Interquartile Range (IQR) method. The numerical attributes considered are Age, StudyTimeWeekly, Absences, and GPA. After detecting outliers, capping is applied to replace extreme values with the calculated lower and upper bounds. Finally, the dataset is checked again to ensure that no outliers remain.

Code for task6: numeric_cols <- c("Age", "StudyTimeWeekly", "Absences", "GPA")

```
for(col in numeric_cols) {
  x <- student_data[[col]]
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_value <- Q3 - Q1
  lower_bound <- Q1 - 1.5 * IQR_value
  upper_bound <- Q3 + 1.5 * IQR_value
  outliers <- student_data[x < lower_bound | x > upper_bound, ]
  cat("\n==============================\n")
  cat("Outliers in column:", col, "\n")
  cat("==============================\n")
  if(nrow(outliers) == 0){
    cat("No outliers found.\n")
```

```
  } else {
    print(outliers)
  }
  student_data[[col]][x < lower_bound] <- lower_bound
  student_data[[col]][x > upper_bound] <- upper_bound
}
outliers <- NULL
for(col in numeric_cols) {
  x <- student_data[[col]]

  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_value <- Q3 - Q1

  lower_bound <- Q1 - 1.5 * IQR_value
  upper_bound <- Q3 + 1.5 * IQR_value
   outliers <- student_data[x < lower_bound | x > upper_bound, ]
   cat("\n============================\n")
  cat("Checking column:", col, "\n")
  cat("============================\n")

  if(nrow(outliers) == 0){
    cat("No outliers found.\n")
  } else {
    print(outliers)
  }
}
```

Sample output of code for task6:

```
============================
Outliers in column: Age
============================
    StudentID Age Gender Ethnicity ParentalEducation StudyTimeWeekly Absences Tutoring ParentalSupport Extracurricular
316      1315  30      0         0                 2       2.1871181        5        1               1               0
338      1337 150      0         0                 1       0.4513717        3        0               2               1
    Sports Music Volunteering      GPA GradeClass
316      1     0            0 2.844039          2
338      0     0            0 2.985810          2
```

Age attribute has 2 outlier values

```
============================
Outliers in column: StudyTimeWeekly
============================
 [1] StudentID         Age              Gender           Ethnicity         ParentalEducation StudyTimeWeekly
 [7] Absences          Tutoring         ParentalSupport  Extracurricular   Sports            Music
[13] Volunteering      GPA              GradeClass
<0 rows> (or 0-length row.names)
```

StudyTimeWeekly attribute has no outlier value

```
============================
Outliers in column: Absences
============================
    StudentID Age Gender Ethnicity ParentalEducation StudyTimeWeekly Absences Tutoring ParentalSupport Extracurricular
634      1633  15      0         0                 1       5.139046      200        0               2               0
    Sports Music Volunteering      GPA GradeClass
634      0     0            0 1.955873          4
```

Absences attribute has 1 outlier values

```
============================
Outliers in column: GPA
============================
 [1] StudentID         Age              Gender           Ethnicity         ParentalEducation StudyTimeWeekly
 [7] Absences          Tutoring         ParentalSupport  Extracurricular   Sports            Music
[13] Volunteering      GPA              GradeClass
<0 rows> (or 0-length row.names)
> |
```

GPA attribute has no outlier value

```
<0 rows> (or 0-length row.names)
> if(nrow(outliers) == 0){
+     cat("No outliers found.\n")
+   } else {
+     print(outliers)
+   }
No outliers found.
> |
```

From output it is seen that all the outlier is fixed and no outlier is found.

Code description of task6: The code identifies and fixes outliers using the IQR method. First, the numerical attributes (Age, StudyTimeWeekly, Absences, and GPA) are selected. For each attribute, the first quartile (Q1), third quartile (Q3), and IQR are calculated. Using these values, lower and upper bounds are determined. Any values outside these bounds are considered outliers and are displayed in the output. To fix the outliers, capping is applied, where values below the lower bound are replaced with the lower bound and values above the upper bound are replaced with the upper bound. After applying this method, the dataset is checked again, and the output shows "No outliers found," which confirms that all outliers have been successfully handled.

Title of the task7: Checking and Fixing Invalid or Noisy Values

Description of the task7: In this task, invalid and noisy values in the dataset are identified and corrected. Each attribute is checked based on its valid range or category. Invalid values are

either corrected or replaced with appropriate values such as mean, median, or mode to ensure data consistency.

Code for task7: **For Age:** invalid_age <- which(!student_data$Age %in% 15:18)

student_data$Age[invalid_age]

invalid_age

student_data$Age[!student_data$Age %in% 15:18] <- NA

mean_age_NA <- mean(student_data$Age, na.rm = TRUE)

mean_age_NA

student_data$Age[is.na(student_data$Age)] <- mean_age_NA

student_data$Age <- as.integer(round(student_data$Age))

invalid_age <- which(!student_data$Age %in% 15:18)

student_data$Age[invalid_age]

For Gender: invalid_gender <- which(!student_data$Gender %in% c(0,1))

invalid_gender

For Ethnicity: invalid_ethnicity <- which(!student_data$Ethnicity %in% 0:3)

student_data$Ethnicity[invalid_ethnicity]

student_data$Ethnicity[!student_data$Ethnicity %in% 0:3] <- NA

mode_eth <- names(which.max(table(student_data$Ethnicity)))

mode_eth

student_data$Ethnicity[is.na(student_data$Ethnicity)] <- mode_eth

invalid_ethnicity <- which(!student_data$Ethnicity %in% 0:3)

student_data$Ethnicity[invalid_ethnicity]

For ParentalEducation: invalid_parent_edu <- which(!student_data$ParentalEducation %in% 0:4)

invalid_parent_edu

For StudyTimeWeekly: invalid_study_time <- student_data$StudyTimeWeekly < 0 | student_data$StudyTimeWeekly > 20

student_data$StudyTimeWeekly[invalid_study_time]

For Absences: invalid_absence <- which(!student_data$Absences %in% 0:30)

invalid_absence

student_data$Absences[invalid_absence]

student_data$Absences[!student_data$Absences %in% 0:30] <- NA

med_absence_in <- median(student_data$Absences, na.rm = TRUE)

med_absence_in

student_data$Absences[is.na(student_data$Absences)] <- med_absence_in

invalid_absence <- which(!student_data$Absences %in% 0:30)

student_data$Absences[invalid_absence]

For Tutoring: invalid_tutoring <- which(!student_data$Tutoring %in% c(0,1))

invalid_tutoring

student_data$Tutoring[invalid_tutoring]

student_data$Tutoring[!student_data$Tutoring %in% c(0,1)] <- NA

mode_Tu <- names(which.max(table(student_data$Tutoring)))

mode_Tu

student_data$Tutoring[is.na(student_data$Tutoring)] <- mode_Tu

invalid_tutoring <- which(!student_data$Tutoring %in% c(0,1))

student_data$Tutoring[invalid_tutoring]

For ParentalSupport: invalid_pa_su <- which(!student_data$ParentalSupport %in% 0:4)

invalid_pa_su

For Extracurricular: invalid_extracurricular <- which(!student_data$Extracurricular %in% c(0,1))

invalid_extracurricular

For Sports: invalid_game <- which(!student_data$Sports %in% c(0,1))

invalid_game

For Music: invalid_song <- which(!student_data$Music %in% c(0,1))

invalid_song

For Volunteering: invalid_volun <- which(!student_data$Volunteering %in% c(0,1))

invalid_volun

For GradeClass: invalid_gradeclass <- which(!student_data$GradeClass %in% 0:4)

invalid_gradeclass

Sample output of code for task7:

For Age:

```
> invalid_age <- which(!student_data$Age %in% 15:18)
> student_data$Age[invalid_age]
[1] 20 20
> invalid_age
[1] 315 337
> student_data$Age[!student_data$Age %in% 15:18] <- NA
> mean_age_NA <- mean(student_data$Age, na.rm = TRUE)
> mean_age_NA
[1] 16.46946
> student_data$Age[is.na(student_data$Age)] <- mean_age_NA
> student_data$Age <- as.integer(round(student_data$Age))
> invalid_age <- which(!student_data$Age %in% 15:18)
> student_data$Age[invalid_age]
integer(0)
>
```

For Gender:

```
> invalid_gender <- which(!student_data$Gender %in% c(0,1))
> invalid_gender
integer(0)
>
```

For Ethnicity:

```
> invalid_ethnicity <- which(!student_data$Ethnicity %in% 0:3)
> student_data$Ethnicity[invalid_ethnicity]
[1] "Other" "Asian" "9"
>
> student_data$Ethnicity[!student_data$Ethnicity %in% 0:3] <- NA
> mode_eth <- names(which.max(table(student_data$Ethnicity)))
> mode_eth
[1] "0"
> student_data$Ethnicity[is.na(student_data$Ethnicity)] <- mode_eth
>
> invalid_ethnicity <- which(!student_data$Ethnicity %in% 0:3)
> student_data$Ethnicity[invalid_ethnicity]
character(0)
>
```

For ParentalEducation:

```
> invalid_parent_edu <- which(!student_data$ParentalEducation %in% 0:4)
> invalid_parent_edu
integer(0)
>
```

For StudyTimeWeekly:

```
> invalid_study_time <- student_data$StudyTimeWeekly < 0 | student_data$StudyTimeWeekly > 20
> student_data$StudyTimeWeekly[invalid_study_time]
numeric(0)
>
```

For Absences:

```
> invalid_absence <- which(!student_data$Absences %in% 0:30)
> invalid_absence
[1] 633
> student_data$Absences[invalid_absence]
[1] 44.5
>
> student_data$Absences[!student_data$Absences %in% 0:30] <- NA
> med_absence_in <- median(student_data$Absences, na.rm = TRUE)
> med_absence_in
[1] 15
> student_data$Absences[is.na(student_data$Absences)] <- med_absence_in
>
> invalid_absence <- which(!student_data$Absences %in% 0:30)
> student_data$Absences[invalid_absence]
numeric(0)
```

For Tutoring:

```
> invalid_tutoring <- which(!student_data$Tutoring %in% c(0,1))
> invalid_tutoring
[1] 35
> student_data$Tutoring[invalid_tutoring]
[1] "11"
>
> student_data$Tutoring[!student_data$Tutoring %in% c(0,1)] <- NA
> mode_Tu <- names(which.max(table(student_data$Tutoring)))
> mode_Tu
[1] "0"
> student_data$Tutoring[is.na(student_data$Tutoring)] <- mode_Tu
>
> invalid_tutoring <- which(!student_data$Tutoring %in% c(0,1))
> student_data$Tutoring[invalid_tutoring]
character(0)
```

For ParentalSupport:

```
> invalid_pa_su <- which(!student_data$ParentalSupport %in% 0:4)
> invalid_pa_su
integer(0)
>
```

For Extracurricular:

```
> invalid_extracurricular <- which(!student_data$Extracurricular %in% c(0,1))
> invalid_extracurricular
integer(0)
>
```

For Sports:

```
> invalid_game <- which(!student_data$Sports %in% c(0,1))
> invalid_game
integer(0)
```

For Music:

```
> invalid_song <- which(!student_data$Music %in% c(0,1))
> invalid_song
integer(0)
>
```

For Volunteering:

```
> invalid_volun <- which(!student_data$Volunteering %in% c(0,1))
> invalid_volun
integer(0)
>
```

For GradeClass:

```
> invalid_gradeclass <- which(!student_data$GradeClass %in% 0:4)
> invalid_gradeclass
integer(0)
>
```

Code description of task7: The code checks each attribute for invalid or noisy values based on its valid range or category. For numerical attributes like Age and Absences, values outside the valid range are identified and replaced with NA, and then filled using mean or median. For categorical attributes like Ethnicity and Tutoring, incorrect values such as strings or invalid numbers are corrected or replaced using mode. Some attributes such as Gender, ParentalEducation, StudyTimeWeekly, ParentalSupport, Extracurricular, Sports, Music, Volunteering, and GradeClass do not contain any invalid values. After applying all corrections, the dataset is checked again, and no invalid values remain.

Title of the task8: Conversion of Attributes from Numeric to Categorical

Description of the task8: In this task, attribute conversion is performed to improve data interpretability. Since categorical attributes were already encoded numerically, a numeric attribute is converted back into categorical form. The Gender attribute is selected for this purpose.

Code for task8:

```
student_data$Gender <- factor(student_data$Gender,
                              levels = c(0,1),
                              labels = c("Male","Female"))
table(student_data$Gender)
```

Sample output of code for task8:

```
> student_data$Gender <- factor(student_data$Gender,
+                               levels = c(0,1),
+                               labels = c("Male","Female"))
> table(student_data$Gender)

  Male Female
  1169   1223
>
```

Code description of task8: The Gender attribute is converted from numeric to categorical form using the factor() function. The numeric values 0 and 1 are replaced with labels “Male” and “Female”. This conversion improves the readability and interpretability of the dataset.

Title of the task9: Filtering the Data

Description of the task9: In this task, a filtering method is applied to select a specific group of students from the dataset. Students with GPA greater than 3 are filtered from the dataset. This helps to focus on a particular subset of students for further analysis.

Code for task9: student_high_gpa <- student_data[student_data$GPA > 3,]

head(student_high_gpa)

Sample output of code for task9:

```
> student_high_gpa <- student_data[student_data$GPA > 3, ]
> head(student_high_gpa)
   StudentID Age Gender Ethnicity Height ParentalEducation StudyTimeWeekly Absences Tutoring ParentalSupport
2       1002  18   Male         0    158                 1       15.408756        0        0               1
6       1006  18   Male         0    149                 1        8.191219        0        0               1
10      1010  16 Female         0    161                 1       18.444466        0        0               3
40      1039  15 Female         1    168                 1        2.949078        3        1               1
46      1045  18 Female         0    173                 1       18.921512        1        1               3
66      1065  17   Male         0    157                 2       16.388557        3        1               4
   Extracurricular Sports Music Volunteering      GPA GradeClass
2                0      0     0            0 3.042915          1
6                1      0     0            0 3.084184          1
10               1      0     0            0 3.573474          0
40               1      1     0            0 3.018906          1
46               1      1     0            0 4.000000          0
66               0      0     0            0 3.347357          1
>
```

Code description of task9: The code applies a filtering condition to the dataset and selects only those students whose GPA is greater than 3. The filtered data is stored in a new data frame named student_high_gpa. The head() function is then used to display the first few rows of the filtered dataset. The output shows that only the records of students with higher GPA values are included in the filtered data.

Title of the task10 : Feature Selection

Description of the task10: In this task, feature selection is performed to identify irrelevant attributes in the dataset. Correlation analysis is used to determine the relationship between selected attributes and GPA. Attributes with very low correlation are considered not useful for analysis.

Code for task10: **For Height**:

correlation_value <- cor(student_data$Height, student_data$GPA, use = "complete.obs")
correlation_value

For StudentID:

correlation_value <- cor(student_data$StudentID, student_data$GPA, use = "complete.obs")
correlation_value

Sample output of code for task10:

For Height:

```
R 4.5.2 · C:/Users/anika/Downloads/Data Science Project/
>
> correlation_value <- cor(student_data$Height, student_data$GPA, use = "complete.obs")
> correlation_value
[1] 0.00212916
```

For StudentID:

```
R 4.5.2 · C:/Users/anika/Downloads/Data Science Project/
> correlation_value <- cor(student_data$StudentID, student_data$GPA, use = "complete.obs")
> correlation_value
[1] -0.002530232
```

Code description of task10: The code performs feature selection using correlation analysis. First, the correlation between Height and GPA is calculated using the cor() function. The result shows that the correlation value is very close to 0, indicating no linear relationship between these variables. Similarly, the correlation between StudentID and GPA is calculated, and it also shows a very low value. Since StudentID is only an identifier and does not contribute meaningful information, and Height has no significant relationship with GPA, these attributes are considered not useful for analysis. However, they are not removed from the dataset to maintain data integrity as per the project requirement.

Title of the task11: Descriptive Statistics According to Target Classes

Description of the task11: In this task, descriptive statistics are calculated for the numerical variables according to the target variable GradeClass. The dataset is grouped by GradeClass, and the average values of GPA, Absences, and StudyTimeWeekly are calculated to examine how these numerical variables differ across different academic performance categories.

Code for task11: **For GPA:** aggregate(GPA ~ GradeClass, data = student_data, mean)

For Absences: aggregate(Absences ~ GradeClass, data = student_data, mean)

For StudyTimeWeekly: aggregate(StudyTimeWeekly ~ GradeClass, data = student_data, mean)

Sample output of code for task11:

For GPA:

```
> aggregate(GPA ~ GradeClass, data = student_data, mean)
  GradeClass      GPA
1          0 3.102942
2          1 3.001673
3          2 2.659742
4          3 2.215000
5          4 1.208041
>
```

For Absences:

```
> aggregate(Absences ~ GradeClass, data = student_data, mean)
  GradeClass  Absences
1          0  5.747664
2          1  5.312268
3          2  7.250639
4          3 11.442029
5          4 20.782824
>
```

For StudyTimeWeekly:

```
> aggregate(StudyTimeWeekly ~ GradeClass, data = student_data, mean)
  GradeClass StudyTimeWeekly
1          0       11.854926
2          1       11.122335
3          2       10.106404
4          3        9.757963
5          4        9.184822
>
```

Code description of task11: The aggregate() function is used to calculate the average values of numerical variables according to the target classes. First, the average GPA is calculated for each GradeClass to observe how academic performance varies across different grade categories. Then, the average number of absences is computed for each GradeClass to analyze the relationship between attendance and performance. Finally, the average weekly study time is calculated to examine how study habits differ among students.

The results show that students in lower GradeClass values (better performance) have higher average GPA and study time, while students in higher GradeClass values (poorer performance) tend to have more absences. This indicates that study habits and attendance are strongly related to academic performance.

Title of the task12: Comparison of Average Values Across Two Categories

Description of the task12: In this task, the average values of selected numerical variables are compared across two distinct categories of a categorical variable. The Gender attribute is selected as the categorical variable, and GPA and StudyTimeWeekly are chosen as numerical variables. This comparison helps to analyze differences in academic performance and study habits between male and female students.

Code for task12:

For GPA: aggregate(GPA ~ Gender, data = student_data, mean)

For StudyTimeWeekly: aggregate(StudyTimeWeekly ~ Gender, data = student_data, mean)

Sample output of code for task12:

For GPA:

```
> aggregate(GPA ~ Gender, data = student_data, mean)
  Gender      GPA
1   Male 1.917807
2 Female 1.894894
>
```

For StudyTimeWeekly:

```
R 4.5.2 · C:/Users/anika/Downloads/Data Science Project/
> aggregate(StudyTimeWeekly ~ Gender, data = student_data, mean)
  Gender StudyTimeWeekly
1   Male        9.702614
2 Female        9.838307
>
```

Code description of task12: The aggregate() function is used to calculate the mean (average) values of GPA and StudyTimeWeekly for each category of the Gender variable. This allows comparison of academic performance and study habits between male and female students. The results show that male students have a slightly higher average GPA compared to female students, although the difference is very small, indicating that gender does not have a significant impact on academic performance. Similarly, the comparison of StudyTimeWeekly shows how study habits vary between the two groups, providing additional insight into student behavior. Overall, the differences between the categories are minimal, suggesting that gender has limited influence on both GPA and study time.

Title of the task13: Examination and Comparison of Variability Across Categories

Description of the task13: In this task, the variability of a numerical variable is examined and compared across different categories of a categorical variable. The Absences attribute is selected as the numerical variable, and GradeClass is chosen as the categorical variable. Measures such as standard deviation, range, and interquartile range (IQR) are used to analyze how attendance varies among students in different grade categories.

Code for task13: aggregate(Absences ~ GradeClass, data = student_data, sd)

tapply(student_data$Absences, student_data$GradeClass, range)

tapply(student_data$Absences, student_data$GradeClass, IQR)

Sample output of code for task13:

```
> aggregate(Absences ~ GradeClass, data = student_data, sd)
  GradeClass Absences
1          0 7.724127
2          1 5.823835
3          2 4.874696
4          3 4.658848
5          4 5.346497
> 
```

```
> tapply(student_data$Absences, student_data$GradeClass, range)
$`0`
[1]  0 29

$`1`
[1]  0 29

$`2`
[1]  0 28

$`3`
[1]  0 29

$`4`
[1]  0 29

> 
```

```
> tapply(student_data$Absences, student_data$GradeClass, IQR)
  0   1   2   3   4
6.5 5.0 5.0 6.0 8.0
>
```

Code description of task13: The aggregate() function is used to calculate the standard deviation of absences for each GradeClass, while the tapply() function is used to compute the range and interquartile range (IQR). These measures help to analyze the variability or spread of absences across different grade categories.

The results show that the standard deviation of absences is highest for GradeClass 0 and gradually decreases for the middle categories, indicating that students in higher-performing groups have more variation in attendance. The range of absences is almost similar across all GradeClass values (mostly from 0 to around 28–29), suggesting that the overall spread of minimum and maximum values is consistent across groups.

However, the IQR values differ across categories, with higher IQR observed in some GradeClass groups (such as GradeClass 3 and 4), indicating greater variability in the middle portion of the data for those groups. In contrast, lower IQR values in other groups suggest more consistent attendance patterns.Overall, the variability of absences differs across grade categories, showing that attendance behavior is not uniform and varies depending on the level of academic performance.

Title of the task 14: Conversion of Imbalanced Dataset into Balanced Dataset using Oversampling

Description of the task 14: In this task, the imbalance in the dataset is addressed using the oversampling technique. The distribution of the target variable GradeClass is examined, and the minority classes are increased by randomly duplicating their samples so that all classes have equal representation. This helps to balance the dataset and ensures that each class contributes equally to the analysis.

Code for task 14:

```
table(student_data$GradeClass)

max_count <- max(table(student_data$GradeClass))

balanced_data <- do.call(rbind, lapply(split(student_data, student_data$GradeClass),
function(x) {

  x[sample(nrow(x), max_count, replace = TRUE), ]

}))

table(balanced_data$GradeClass)
```

Sample output of code for task14:

Before OverSampling:

```
> table(student_data$GradeClass)

   0    1    2    3    4
 107  269  391  414 1211
>
```

After OverSampling:

```
> max_count <- max(table(student_data$GradeClass))
> balanced_data <- do.call(rbind, lapply(split(student_data, student_data$GradeClass), function(x) {
+   x[sample(nrow(x), max_count, replace = TRUE), ]
+ }))
> table(balanced_data$GradeClass)

   0    1    2    3    4
1211 1211 1211 1211 1211
>
```

Code description of task14: The table() function is first used to examine the distribution of the GradeClass variable, which shows that the dataset is imbalanced. To address this issue, oversampling is applied by identifying the maximum class size and then randomly duplicating samples from smaller classes using replacement. This process ensures that all classes have equal representation in the dataset.The results show that after applying oversampling, the dataset becomes balanced, as each GradeClass contains an equal number of observations. This improves the quality of the dataset and helps to avoid bias towards majority classes in further analysis.

Title of the task15: Normalization of Continuous Attribute

Description of the task15: In this task, normalization is applied to a continuous attribute of the balanced dataset. The GPA attribute is normalized using the Min-Max normalization method so that all values fall within the range of 0 to 1. This helps to bring the values into a common scale for further analysis.

Code for task15: balanced_data$GPA_norm <- (balanced_data$GPA - min(balanced_data$GPA)) / (max(balanced_data$GPA) - min(balanced_data$GPA))

head(balanced_data$GPA_norm)

Sample output of code for task15:

```
> balanced_data$GPA_norm <- (balanced_data$GPA - min(balanced_data$GPA)) /
+   (max(balanced_data$GPA) - min(balanced_data$GPA))
> head(balanced_data$GPA_norm)
[1] 0.5448393 0.8901979 1.0000000 1.0000000 0.1441007 0.5792400
>
```

Code description of task15: The GPA attribute of the balanced dataset is normalized using the Min-Max normalization method. The formula subtracts the minimum GPA value from each GPA and then divides by the difference between the maximum and minimum GPA values. As a result, all normalized GPA values fall between 0 and 1. This transformation makes the data easier to compare on a common scale.

Title of the task16: Splitting the Dataset for Training and Testing

Description of the task16: In this task, the balanced dataset is divided into training and testing datasets. A total of 70% of the data is used for training and the remaining 30% is used for testing. This split helps prepare the dataset for future model building and evaluation.

Code for task16: set.seed(123)

train_index <- sample(1:nrow(balanced_data), 0.7 * nrow(balanced_data))

train_data <- balanced_data[train_index,]

test_data <- balanced_data[-train_index,]

nrow(train_data)

nrow(test_data)

Sample output of code for task16:

```
> set.seed(123)
> train_index <- sample(1:nrow(balanced_data), 0.7 * nrow(balanced_data))
> train_data <- balanced_data[train_index, ]
> test_data <- balanced_data[-train_index, ]
> nrow(train_data)
[1] 4238
> nrow(test_data)
[1] 1817
>
```

Code description of task16: The set.seed(123) function is used to ensure that the random sampling process gives the same result each time the code is run. Then, 70% of the row indices of the balanced dataset are randomly selected and stored in train_index. Using these indices, the dataset is split into two parts: train_data for training and test_data for testing. Finally, the nrow() function is used to show the number of rows in each dataset. The output confirms that the dataset has been successfully divided into training and testing subsets.

Project Code

Full Project Code Given Below:

```
student_data <- read.csv("ids_mid_project_group_05.csv") head(student_data)
str(student_data)

summary(student_data)
dim(student_data)
names(student_data)

colSums(is.na(student_data))

student_data$Age[is.na(student_data$Age)] <- mean(student_data$Age, na.rm = TRUE)
student_data$Age <- as.integer(round(student_data$Age))
sum(is.na(student_data$Age))

mode_gender <- names(which.max(table(student_data$Gender)))
mode_gender
student_data$Gender[is.na(student_data$Gender)] <- mode_gender
sum(is.na(student_data$Gender))

mode_parent_edu <- names(which.max(table(student_data$ParentalEducation)))
mode_parent_edu
student_data$ParentalEducation[is.na(student_data$ParentalEducation)] <- mode_parent_edu
sum(is.na(student_data$ParentalEducation))

med_absence <- median(student_data$Absences, na.rm = TRUE)
med_absence
student_data$Absences[is.na(student_data$Absences)] <- med_absence
sum(is.na(student_data$Absences))
```

```
mode_tutoring <- names(which.max(table(student_data$Tutoring)))
mode_tutoring
student_data$Tutoring[is.na(student_data$Tutoring)] <- mode_tutoring
sum(is.na(student_data$Tutoring))

mode_parent_sup <- names(which.max(table(student_data$ParentalSupport)))
mode_parent_sup
student_data$ParentalSupport[is.na(student_data$ParentalSupport)] <- mode_parent_sup
sum(is.na(student_data$ParentalSupport))

mode_eca <- names(which.max(table(student_data$Extracurricular)))
mode_eca
student_data$Extracurricular[is.na(student_data$Extracurricular)] <- mode_eca
sum(is.na(student_data$Extracurricular))

mode_sports <- names(which.max(table(student_data$Sports)))
mode_sports
student_data$Sports[is.na(student_data$Sports)] <- mode_sports
sum(is.na(student_data$Sports))

mean_gpa <- mean(student_data$GPA, na.rm = TRUE)
mean_gpa
student_data$GPA[is.na(student_data$GPA)] <- mean_gpa
sum(is.na(student_data$GPA))

anyDuplicated(student_data)
student_data <- student_data[!duplicated(student_data), ]

numeric_cols <- c("Age", "StudyTimeWeekly", "Absences", "GPA")
```

```
for(col in numeric_cols) {
  x <- student_data[[col]]

  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_value <- Q3 - Q1

  lower_bound <- Q1 - 1.5 * IQR_value
  upper_bound <- Q3 + 1.5 * IQR_value

  outliers <- student_data[x < lower_bound | x > upper_bound, ]

  cat("\n==============================\n")
  cat("Outliers in column:", col, "\n")
  cat("==============================\n")

  if(nrow(outliers) == 0){
    cat("No outliers found.\n")
  } else {
    print(outliers)
  }

  student_data[[col]][x < lower_bound] <- lower_bound
  student_data[[col]][x > upper_bound] <- upper_bound
}

outliers <- NULL

for(col in numeric_cols) {
```

```
  x <- student_data[[col]]

  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_value <- Q3 - Q1

  lower_bound <- Q1 - 1.5 * IQR_value
  upper_bound <- Q3 + 1.5 * IQR_value

  outliers <- student_data[x < lower_bound | x > upper_bound, ]

  cat("\n============================\n")
  cat("Checking column:", col, "\n")
  cat("============================\n")

  if(nrow(outliers) == 0){
    cat("No outliers found.\n")
  } else {
    print(outliers)
  }
}

invalid_age <- which(!student_data$Age %in% 15:18)
student_data$Age[invalid_age]
invalid_age

student_data$Age[!student_data$Age %in% 15:18] <- NA
mean_age_NA <- mean(student_data$Age, na.rm = TRUE)
mean_age_NA
student_data$Age[is.na(student_data$Age)] <- mean_age_NA
```

```
student_data$Age <- as.integer(round(student_data$Age))

invalid_age <- which(!student_data$Age %in% 15:18)
student_data$Age[invalid_age]

invalid_gender <- which(!student_data$Gender %in% c(0,1))
invalid_gender

invalid_ethnicity <- which(!student_data$Ethnicity %in% 0:3)
student_data$Ethnicity[invalid_ethnicity]

ethnicity_map <- c(
  "Caucasian" = 0,
  "African American" = 1,
  "Asian" = 2,
  "Other" = 3
)

student_data$Ethnicity <- ifelse(
  student_data$Ethnicity %in% names(ethnicity_map),
  ethnicity_map[student_data$Ethnicity],
  student_data$Ethnicity
)

invalid_ethnicity <- which(!student_data$Ethnicity %in% 0:3)
student_data$Ethnicity[invalid_ethnicity]

student_data$Ethnicity[!student_data$Ethnicity %in% 0:3] <- NA
mode_eth <- names(which.max(table(student_data$Ethnicity)))
mode_eth
```

```
student_data$Ethnicity[is.na(student_data$Ethnicity)] <- mode_eth

invalid_ethnicity <- which(!student_data$Ethnicity %in% 0:3)
student_data$Ethnicity[invalid_ethnicity]

invalid_parent_edu <- which(!student_data$ParentalEducation %in% 0:4)
invalid_parent_edu

invalid_study_time <- student_data$StudyTimeWeekly < 0 | student_data$StudyTimeWeekly
> 20
student_data$StudyTimeWeekly[invalid_study_time]

invalid_absence <- which(!student_data$Absences %in% 0:30)
invalid_absence
student_data$Absences[invalid_absence]

student_data$Absences[!student_data$Absences %in% 0:30] <- NA
med_absence_in <- median(student_data$Absences, na.rm = TRUE)
med_absence_in
student_data$Absences[is.na(student_data$Absences)] <- med_absence_in

invalid_absence <- which(!student_data$Absences %in% 0:30)
student_data$Absences[invalid_absence]

invalid_tutoring <- which(!student_data$Tutoring %in% c(0,1))
invalid_tutoring
student_data$Tutoring[invalid_tutoring]

student_data$Tutoring[!student_data$Tutoring %in% c(0,1)] <- NA
mode_Tu <- names(which.max(table(student_data$Tutoring)))
mode_Tu
```

```
student_data$Tutoring[is.na(student_data$Tutoring)] <- mode_Tu

invalid_tutoring <- which(!student_data$Tutoring %in% c(0,1))
student_data$Tutoring[invalid_tutoring]

invalid_pa_su <- which(!student_data$ParentalSupport %in% 0:4)
invalid_pa_su

invalid_extracurricular <- which(!student_data$Extracurricular %in% c(0,1))
invalid_extracurricular

invalid_game <- which(!student_data$Sports %in% c(0,1))
invalid_game

invalid_song <- which(!student_data$Music %in% c(0,1))
invalid_song

invalid_volun <- which(!student_data$Volunteering %in% c(0,1))
invalid_volun

invalid_gradeclass <- which(!student_data$GradeClass %in% 0:4)
invalid_gradeclass

student_data$Gender <- factor(student_data$Gender,
                              levels = c(0,1),
                              labels = c("Male","Female"))
table(student_data$Gender)

student_high_gpa <- student_data[student_data$GPA > 3, ]
head(student_high_gpa)
```

```
correlation_value <- cor(student_data$Height, student_data$GPA, use = "complete.obs")
correlation_value

correlation_value <- cor(student_data$StudentID, student_data$GPA, use = "complete.obs")
correlation_value

aggregate(GPA ~ GradeClass, data = student_data, mean)
aggregate(Absences ~ GradeClass, data = student_data, mean)
aggregate(StudyTimeWeekly ~ GradeClass, data = student_data, mean)

aggregate(GPA ~ Gender, data = student_data, mean)
aggregate(StudyTimeWeekly ~ Gender, data = student_data, mean)

aggregate(Absences ~ GradeClass, data = student_data, sd)
tapply(student_data$Absences, student_data$GradeClass, range)
tapply(student_data$Absences, student_data$GradeClass, IQR)

table(student_data$GradeClass)
max_count <- max(table(student_data$GradeClass))
balanced_data <- do.call(rbind, lapply(split(student_data, student_data$GradeClass),
function(x) {
  x[sample(nrow(x), max_count, replace = TRUE), ]
}))
table(balanced_data$GradeClass)

balanced_data$GPA_norm <- (balanced_data$GPA - min(balanced_data$GPA)) /
  (max(balanced_data$GPA) - min(balanced_data$GPA))
head(balanced_data$GPA_norm)

set.seed(123)
```

```
train_index <- sample(1:nrow(balanced_data), 0.7 * nrow(balanced_data))
train_data <- balanced_data[train_index, ]
test_data <- balanced_data[-train_index, ]
nrow(train_data)
nrow(test_data)
```